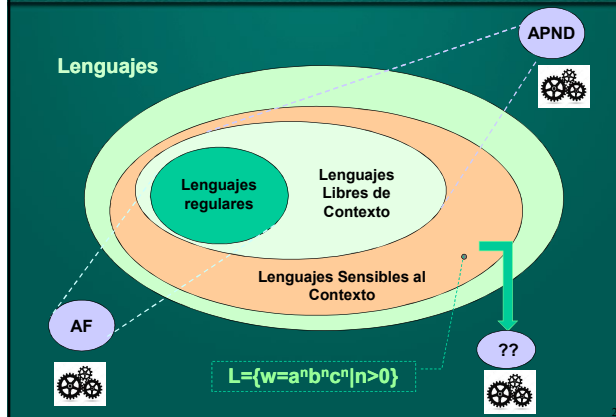


Teoría de la Computabilidad

Módulo 8: Introducción a las Máquinas de Turing

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Extendiendo el poder de cómputo



Extendiendo el poder de cómputo

- Los lenguajes sensibles al contexto tienen como formalismo reconocedor a los **Autómatas Acotados Linealmente (AAL)**
- No nos detendremos a estudiar este autómata (pues puede verse como restricción de otro formalismo que es más potente aún...)
- Dado que hemos incrementado nuestro poder de cómputo lentamente, y entendemos mejor el concepto de autómata, ahora seremos más ambiciosos...

Extendiendo el poder de cómputo

¿Existe un autómata que reconozca cualquier lenguaje de los que vimos anteriormente?

Hasta el momento, cada autómata que estudiábamos no era capaz de reconocer algún lenguaje.

- Los AF se veían limitados ante $L=\{a^n b^n \mid n>0\}$
- Los APND se veían limitados ante $L=\{a^n b^n c^n \mid n>0\}$

Vale la pena preguntarse si se repetirá esta situación una vez más...

¿Existe algún lenguaje que este nuevo autómata no pueda reconocer?

De ser así,
¿puede ser reemplazado por otro más poderoso?

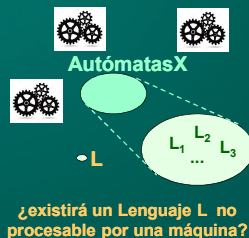
Extendiendo el poder de cómputo

¿Que significaría definir una clase de autómatas que reconozca los lenguajes anteriores...

... y que **exista** un lenguaje **L** que no pueda reconocer...

... y que **no** pueda construirse otro autómata más potente?...

Pregunta central en computación: ¿qué máquinas puedo construir? ¿qué límites tendrán?



Extendiendo el poder de cómputo

Recordemos que los autómatas siempre computan una función (usualmente estudiamos la de *reconocer* una cadena).

En la primera pregunta, habremos encontrado una forma de realizar cualquier cálculo sobre cualquier lenguaje, **sin limitaciones**.

En la segunda pregunta, habremos encontrado una forma de realizar cualquier cálculo sobre ciertos lenguajes, y habremos descubierto que **hay cosas que nunca podremos resolver**.

¡Responder a estas preguntas es encontrar el límite de la computabilidad!

Computabilidad

Dos científicos abordaron especialmente este problema:



Alan Turing
(23/6/1912 - 7/6/1954)



Alonzo Church
(14/6/1903 - 11/8/1995)



Películas noveladas sobre Turing:
“El Código Enigma” (The Imitation Game,
2014) Y “Enigma” (2001)

Una breve Evolución Histórica

Hacia fines de siglo XIX, el matemático D. Hilbert postuló la necesidad de bases firmes para las matemáticas. Con este fin, en un congreso realizado en 1900 se plantearon tres problemas:

Dada una afirmación cualquiera de la matemática, ¿puede demostrarse siempre si ésta es verdadera o falsa?

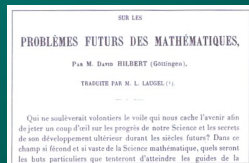
La matemática ¿es consistente?

“Entscheidungsproblem” (en alemán = “problema de la decisión”): ¿existen métodos para responder a la primer pregunta?

Una breve Evolución Histórica

Hacia fines de siglo XIX, el matemático alemán David Hilbert postuló la necesidad de bases firmes para las matemáticas. Con este fin, en un congreso realizado en 1900 se plantearon tres problemas (reescritos simplifícadamente):

- 1 - ¿Se puede demostrar cualquier cosa?
- 2 - Lo demostrable en un sistema dado: ¿es siempre consistente?
- 3 - ¿Siempre existen métodos para demostrar cualquier cosa en un sistema dado?



David Hilbert
(1862-1943)

Una breve Evolución Histórica

En un nuevo congreso en 1928, el matemático checo Kurt Gödel demostró que algunos de los problemas planteados por Hilbert tenían respuesta **negativa**.

Toda teoría formal con suficiente poder expresivo como para reproducir la aritmética tenía teoremas no demostrables, o bien era inconsistente.

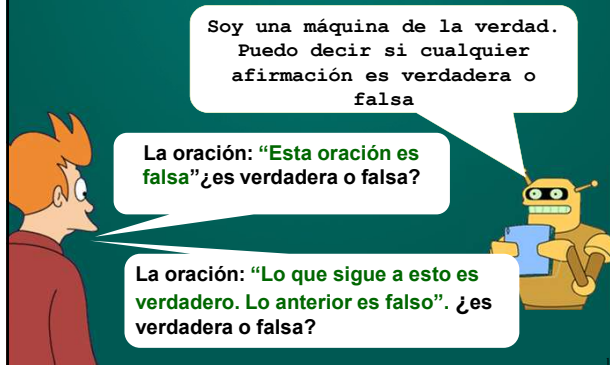
Este hecho tuvo consecuencias tremendas sobre las matemáticas...



Kurt Gödel
(nació el 28 Abril de 1906 en Brno, Imperio Austrohúngaro; murió el 14 de enero de 1978 en New Jersey, USA)

Una breve Evolución Histórica

Breve ejemplo de lo que afirma Gödel:



Entscheidungsproblem

Pero el “problema de la decisión” seguía pendiente, y atrajo el interés del estudiante Alan Turing (en Cambridge, R.Unido) y Alonzo Church (EEUU).

Turing y Church mostraron separadamente que existen problemas sobre los que nunca podrá establecerse un método para resolverlos.

Indirectamente, el acercamiento de Turing involucraba cómo debía definirse la noción de **método o procedimiento efectivo**, que llegó a nuestros días como **algoritmo**.

Esto resultó en un modelo formal, llamado **Máquina de Turing**, que establece qué problemas puede resolver una computadora actual.

Procedimiento Efectivo (p.e.)

Def: Un **procedimiento efectivo (p.e.)** es un conjunto de reglas, destinadas a resolver un problema, escritas en un determinado lenguaje, que son interpretables y ejecutables.

La noción de **procedimiento efectivo** es primitiva

(no está definida por otros conceptos)

Máquinas de Turing
Funciones recursivas parciales (Kleene, 1936)
Gramáticas estructuradas por frases (Chomsky, 1956)
Redes de Petri (Petri, 1962)

13

Procedimientos vs. Algoritmos

En nuestro enfoque, no distinguiremos entre **procedimiento efectivo** y **algoritmo**.

Si distinguiremos entre algoritmos totales y parciales.

- **Algoritmo total:** siempre otorga una respuesta.
- **Algoritmo parcial:** no necesariamente otorga una respuesta.

14

Propiedades Básicas de Algoritmos

- Un **P.E.** está formado por una secuencia finita de instrucciones.
- Existe un procesador mecánico o agente que puede interpretar las instrucciones, y producir resultados predecibles y repetibles.
- El procesador puede almacenar resultados intermedios en una memoria.
- No existe un límite finito ni para la entrada y para la salida de datos.
- No existe límite a la cantidad de almacenamiento requerido para realizar la computación.
- No existe límite a la cantidad de pasos discretos requeridos. *Pueden existir computaciones infinitas.*

15

Máquinas de Turing

En 1936 Turing (quien tenía entonces 24 años) definió una máquina abstracta, conocida hoy como **Máquina de Turing**.

Su idea central era abstraer el conjunto de operaciones que realiza una persona cuando realiza un cálculo.

El gran aporte de Turing es establecer que una máquina de Turing es capaz de ejecutar cualquier procedimiento efectivo.

¿Qué es un algoritmo? ¿Qué significa "computar" algo...?



16

Un resultado trascendental

Todos los algoritmos que existen en el universo

Máquinas de Turing

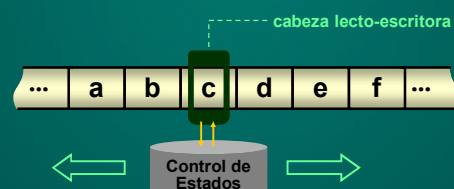
¡Creo que una MT puede resolver CUALQUIER algoritmo!



Si existe un algoritmo para realizar cierta tarea, entonces también existe una Máquina de Turing que realice la misma tarea, y viceversa.

17

Máquina de Turing: Idea



La cabeza lecto-escritora puede **leer** y **escribir** símbolos sobre la cinta.

La cabeza lecto-escritora puede realizar movimientos hacia la **izquierda** o hacia la **derecha**.

18

Máquina de Turing

Def.: Una MT es una 5-upla $T=(S, \Sigma, \Gamma, \delta, s_0, F)$ donde:

S es el conjunto de estados, S finito, $S \neq \emptyset$

Σ es el alfabeto de trabajo - Γ alfabeto de entrada

δ es una función parcial,

$$\delta: S \times \Sigma \rightarrow S \times \Sigma \times \{ \rightarrow, \leftarrow, - \}$$

donde $\rightarrow$ =mov.der $\leftarrow$ =mov.izq, $-$ =no mover

s_0 es el estado inicial, $s_0 \in S$.

F es el conjunto de estados finales, $F \subseteq S$.

19

Convenciones

- 1) Se asume que la cinta contiene inicialmente asteriscos (*), y la cadena de entrada está encerrada entre #.

Por ejemplo, si la cadena es **abc**, la cinta contiene:

#**abc**

Luego $\Sigma \supset \{ *, \# \}$

20

Convenciones

- 2) Inicialmente la cabeza lectoescritora está ubicada sobre el 1er. caracter de la entrada, es decir el primer caracter no blanco de la cinta después de #.

* * * # **a** b c d # * * * *

- 3) Combinaremos operaciones de escritura y desplazamiento para abreviar algunas definiciones.

- 4) Los números usualmente se escribirán en formato unario.
Un número natural n se escribe como una cadena de $n+1$ símbolos "1".

21

Convenciones: el formato unario

Algunas veces usaremos una representación denominada **formato unario** para nros. naturales.

Nro natural n = una cadena de $n+1$ símbolos "1".

Ejemplos:

Si la cinta contiene ***#**1******, esto representa al número natural 0.

Si la cinta contiene ***#**11******, esto representa el número natural 1.

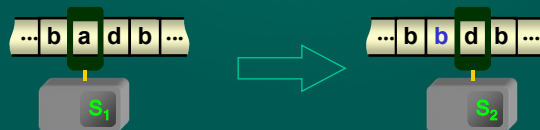
Si la cinta contiene ***#**11111******, esto representa al número natural 4.

22

Idea Intuitiva: cómo funcionan las MTs

δ es una fc. parcial, $\delta: S \times \Sigma \rightarrow S \times \Sigma \times \{ \rightarrow, \leftarrow, - \}$.

$(S_1, a, S_2, b, \rightarrow)$ { Si el estado actual es S_1 , el símbolo bajo la cabeza de la MT es a , entonces pasar al estado S_2 , escribir en la cinta el símbolo b , y mover la cabeza a la derecha.

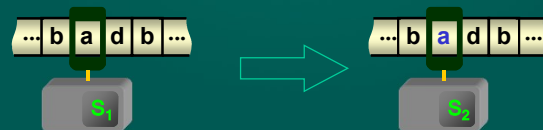


23

Idea Intuitiva: cómo funcionan las MTs

δ es una fc. parcial, $\delta: S \times \Sigma \rightarrow S \times \Sigma \times \{ \rightarrow, \leftarrow, - \}$

$(S_1, a, S_2, a, -)$ { Si el estado actual es S_1 , el símbolo bajo la cabeza de la MT es a , entonces pasar al estado S_2 , escribir en la cinta el símbolo a , y detenerse



24

Representación Gráfica de una MT

En la práctica utilizaremos una representación de MT por medio de grafos, de forma similar a los autómatas a pila.

Cada estado será representado por un nodo del grafo. La función δ se representará por medio de arcos etiquetados con: el símbolo que está bajo la cabeza, seguido del símbolo a escribir y el movimiento realizar.

En general una quintupla $(s_i, \alpha, \beta, s_j, M) \in \delta$ donde $s_i, s_j \in S$, $\alpha, \beta \in \Sigma$, y $M \in \{\rightarrow, \leftarrow, -\}$ se representa en el grafo de la sig. forma:



Los estados aceptadores se identificarán con un doble círculo: $\odot s_k$ si $s_k \in F$

23

Algunos Usos de las Máquinas de Turing

Máquinas de Turing

Para reconocer lenguajes

Para computar funciones

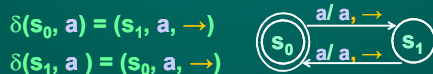
24

Ejemplos de Máquinas de Turing

Desarrollar una MT para reconocer cadenas de $L = \{a^{2n} | n \geq 0\}$.

Sea $T = (S, \Sigma, \Gamma, \delta, s_0, F)$ una MT, donde

$S = \{s_0, s_1\}$, $\Gamma = \{a\}$, $\Sigma = \{a, \#, *\}$, $F = \{s_0\}$, y la fc. δ se define por:

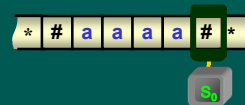


Esta MT resuelve el problema de reconocer L.

Veamos cómo se comporta T para la cadena aaaa...

27

Ejemplos de Máquinas de Turing

 $\delta(s_0, a) = (s_1, a, \rightarrow)$ $\delta(s_1, a) = (s_0, a, \rightarrow)$ 

28

Ejemplos de Máquinas de Turing

Desarrollar una MT para calcular $f(x,y)=x+y$, donde x e y están en formato unario, y se asume que la cinta contiene

...*****#x₁#y₁*****..

donde x₁ e y₁ son las representaciones unarias de x e y.

Ejemplo: 3+1 sería

...*****#1111#11*****..

y el resultado debe ser 4:

...*****#11111#*****..

29

Ejemplos de Máquinas de Turing

...***#11111#***.. ----- ...***#11111#***..

0 + 4

4

...***#1111#11111#***.. ----- ...***#11111111#***..

3 + 4

7

...***#11111...111111#11111...11111#***..

...***##111...111111111111...11111#***..

30

32

34

Turing Machine Simulator

Diferencias con la notación del apunte

- Los estados de la MT se representan con números (1,2,3 ...). No se puede usar el 0.
- Avanzar a derecha o a izquierda y se denota $>$ y $<$
- Existe un estado especial (H) al llegar al cual la MT actúa y luego se detiene.
- El soft incluye algunos ejemplos de MTs.

37

Configuración en MT

Def.: Una **configuración** de una MT $T=(S, \Sigma, \Gamma, \delta, s_0, F)$ es una terna (s, α, i) , donde s es el estado actual de T , $\alpha \in \Sigma^*$, e $i \in \mathbb{Z}^+$ que marca la posición donde está la cabeza.

Nota: adoptando $i \in \mathbb{Z}^+$ estamos usando la cinta en una única dirección. Esto no cambia la capacidad expresiva de la máquina de Turing

38

Transición en MT

Def.: Una **transición** de P es representada con una relación binaria “|-” entre configuraciones:

$$(s, \gamma_1 \alpha \gamma_2, i) \vdash (s', \gamma_1 \alpha' \gamma_2, i')$$

si existe en T una regla de transición

$$\delta(s, \alpha) = (s', \alpha', M)$$

donde $s, s' \in S$; $\alpha, \alpha' \in \Sigma$; $\gamma_1, \gamma_2 \in \Sigma^*$; $M \in \{\rightarrow, \leftarrow, -\}$, y

$$i' = \begin{cases} i-1 & \text{si } M = \leftarrow \\ i & \text{si } M = - \\ i+1 & \text{si } M = \rightarrow \end{cases}$$

39

Aceptación de una cadena en MT

Def.: Una cadena w es **aceptada** o **reconocida** por una máquina $T=(S, \Sigma, \delta, s_0, F)$ si

$$(s_0, w, 1) \vdash (s_f, \alpha, i)$$

para algún $s_f \in F$, $w, \alpha \in \Sigma^*$, $i \in \mathbb{Z}^+$.

Def.: Dado un alfabeto Σ y un lenguaje $L \subseteq \Sigma^*$, L es **aceptado por una MT T** si

$$L = L(T) = \{w \mid w \in \Sigma^* \text{ y } w \text{ es aceptada por } T\}$$

En tal caso diremos que L es **Turing Acceptable** (o **Aceptable por una Máquina de Turing**).

40

Ejemplo más complejo de MT

Desarrollar una MT para reconocer cadenas de $L = \{wcw \mid w \in \{a,b\}^*\}$.

Idea general:

Tachar un símbolo de la primer aparición de w .

Ir a la segunda aparición de w (después de c) y tachar el mismo símbolo

Volver a la primer aparición de w , al símbolo siguiente al recién tachado.

Continuar el proceso hasta tachar los símbolos de las dos apariciones de w .

41

Ejemplo más complejo de MT

Diagram illustrating the process of marking symbols in a string wcw to recognize the language $L = \{wcw \mid w \in \{a,b\}^*\}$. The string is shown with markers $*$ and $\#$ at the beginning and end. The process involves marking the first occurrence of w , then the second occurrence (after c), and so on, until all symbols of w in both occurrences are marked.

42

Ejemplo más complejo de MT

Sea $T=(S, \Sigma, \Gamma, \delta, s_0, F)$ una MT, donde

$S = \{s_0, s_1, s_2, s_3, s_a, s_b, s_{ac}, s_{bc}, s_{parar}\},$

$\Sigma = \{a, b, c, \#, *\},$

$\Gamma = \{a, b, c\},$

$F = \{s_{parar}\}$

Tachamos a o b

$\delta(s_0, a) = (s_a, *, \rightarrow)$

$\delta(s_0, b) = (s_b, *, \rightarrow)$

$\delta(s_0, c) = (s_3, c, \rightarrow)$

Busco c despues de tachar a

$\delta(s_a, a) = (s_a, a, \rightarrow)$

$\delta(s_a, b) = (s_a, b, \rightarrow)$

$\delta(s_a, c) = (s_{ac}, c, \rightarrow)$

Busco c despues de tachar b

$\delta(s_b, a) = (s_b, a, \rightarrow)$

$\delta(s_b, b) = (s_b, b, \rightarrow)$

$\delta(s_b, c) = (s_{bc}, c, \rightarrow)$

Ejemplo más complejo de MT

Desde c, ir a D, hasta hallar a y tachar

$\delta(s_{ac}, *) = (s_{ac}, *, \rightarrow)$

$\delta(s_{ac}, a) = (s_1, *, \leftarrow)$

Desde c, ir a D, hasta hallar b y tachar

$(s_{bc}, *) = (s_{bc}, *, \rightarrow)$

$\delta(s_{bc}, b) = (s_1, *, \leftarrow)$

Ir a izq. hasta hallar una c

$\delta(s_1, *) = (s_1, *, \leftarrow)$

$\delta(s_1, c) = (s_2, c, \leftarrow)$

Ir a izq. hasta hallar comienzo cadena

$\delta(s_2, a) = (s_2, a, \leftarrow)$

$\delta(s_2, b) = (s_2, b, \leftarrow)$

$\delta(s_2, *) = (s_0, *, \rightarrow)$

Verificar que la 2da. cadena también terminó.

$(s_3, *) = (s_3, *, \rightarrow)$

$\delta(s_3, \#) = (s_{parar}, \#, -)$

Ejemplo más complejo de MT

Verificar que la máquina anterior, para la entrada #abcab#, termina en la configuración

$(s_{parar}, \#\#\#c\#\#, 6)$

Verificar que la máquina anterior, para la entrada #abcbb#, termina en la configuración

$(s_{ac}, \#\#bcb\#, 4)$

Ejercicio propuesto

Desarrollar una MT para reconocer cadenas de

$L = \{0^n 10^n 10^n | n \geq 1\}.$

En honor a Kurt Gödel y Alan Turing...

Premio Gödel: se concede anualmente; son 5000 us\$ para el mejor trabajo en teoría de ciencias de la computación.

<http://sigact.acm.org/prizes/godel/>

Premio Turing: se lo conoce como el Premio Nobel de la Computación. El sponsor es Intel Corporation, y el premio son 100.000 us\$.

http://en.wikipedia.org/wiki/Turing_Award

Qué hemos visto en este módulo

- Máquinas de Turing: Evolución histórica.
- Procedimiento Efectivo: noción
- Máquina de Turing: definición formal. Usos de MTs.
- Máquinas de Turing: Configuración. Transición. Aceptación de cadenas. Ejemplos